

Session 6: Line Clearing & Scoring & Level Design

#1 Line Clearing

```
void LineClearing() {
    int lines_cleared = 0;
    for (int row = ROWS-1; row >= 0; row--) {
        bool fullLine = true;
        for (int col = 0; col < COLUMNS; col++) {
            if (board[row][col] == 0) {
                fullLine = false;
                break;
            }
        }
        if (fullLine) {
            for (int r = row; r > 0; r--) {
                for (int c = 0; c < COLUMNS; c++) {
                    board[r][c] = board[r - 1][c];
                }
            }
            for (int c = 0; c < COLUMNS; c++) {
                board[0][c] = 0;
            }
            lines_cleared++;
            row++;
        }
    }
}
```

LineClearing 함수 정의, 화면 기준 아래부터 한 줄이 모두 채워져 있는지 확인하고 채워져 있으면 그 줄을 지우고 가장 윗줄은 빈칸으로 채우기. 줄이 지워지면 한 줄이 내려오기 때문에 row++ 를 해서 다시 확인

#2 Scoring

```
int score; // score
```

전역변수에 score 정의, InitializeGame 함수에 0으로 초기화하는 코드 추가

```
switch (lines_cleared) {
    case 1:
        score += 100;
        break;
    case 2:
        score += 300;
        break;
    case 3:
        score += 500;
```

```

        break;
    case 4:
        score += 800;
        break;
}

```

LineClearing 함수에 계속 추가. lines_cleared 변수로 지워진 줄 수 확인해서 switch 문 돌려서 점수 획득. (점수 출력은 뒤에서)

#3 Level Design

```

int level = 1; // level
int fall_interval = 500; // fall speed

```

전역변수 level 정의, 기존에 상수로 만들어놓은 BLOCK_FALL_INTERVAL 을 전역변수 fall_interval 로 바꿈. InitializeGame 함수에서 각각 1, 500으로 초기화하는 코드 추가

```

static const int score_theshold[3] = {500, 1000, 1500};
if (lines_cleared) {
    printf("score : %d\n", score);
    for (int i = 0; i < 3; i++) {
        if (score >= score_theshold[i]) {
            level = i + 2;
            fall_interval = 500 - (level - 1) * 100;
        }
    }
}
}
}

```

LineClearing 함수에 계속 추가. score_theshond 를 상수로 정의해놓음. 지워진 줄이 있으면 점수를 출력하고, 점수와 score_theshold 를 비교하여 값을 넘으면 level 이 증가하고, 그에 따라 fall_interval 도 감소해서 블록이 더 빨리 떨어짐.